

FDS Assignment Report

Normal-3: Dijkstra Sequence

Course: Fundamentals of Data Structures

Zhejiang University

Date of Completion: May 4, 2026

Chapter 1 Introduction

Dijkstra's algorithm, introduced by Edsger W. Dijkstra in 1956 and published in 1959, is a landmark greedy algorithm that solves the single-source shortest path (SSSP) problem on graphs with non-negative edge weights. Given a source vertex, the algorithm iteratively selects the unvisited vertex with the currently smallest tentative distance from the source, marks it as finalized, and relaxes all of its outgoing edges. This process continues until every reachable vertex has been finalized.

The order in which vertices are finalized forms a sequence we call the Dijkstra sequence. Because ties in tentative distance may be broken arbitrarily, a single graph can admit multiple valid Dijkstra sequences from the same source. The problem in this project is to determine, for each of K given vertex permutations, whether it could legitimately be produced by some execution of Dijkstra's algorithm starting from its first element.

Specifically, the problem requires:

- Reading a weighted, undirected, connected graph with $N_v \leq 10^3$ vertices and $N_e \leq 10^5$ edges.
- Processing $K \leq 100$ query sequences, each a permutation of all N_v vertices.
- For each query, printing "Yes" if the sequence is a valid Dijkstra sequence, or "No" otherwise.

The central challenge is to formalise the notion of a "valid Dijkstra sequence" so that it can be checked efficiently without enumerating all possible Dijkstra executions. The key insight, derived from the greedy choice property, is that a sequence is valid if and only if, at every step i , the i -th vertex has a shortest distance from the source that is no greater than the shortest distance of any vertex that appears later in the sequence. This condition is necessary and sufficient: it precisely captures when each choice could have been the greedy minimum at that moment, accounting for ties.

Chapter 2 Algorithm Specification

2.1 Data Structures

The following data structures are used throughout the program:

- Adjacency matrix `adj[MAXV][MAXV]` (integer array, $\text{MAXV} = 1005$): `adj[u][v]` stores the weight of the direct edge between vertices u and v , or 0 if no such edge exists. When multiple edges (parallel edges) connect the same pair of vertices, only the minimum weight is stored. Memory: $O(Nv^2) \approx 4 \text{ MB}$.
- Distance array `dist[MAXV]`: `dist[v]` holds the current shortest-path distance from the source to vertex v . Initialised to `INF` ($\text{INT_MAX} \gg 1$) except `dist[source] = 0`.
- Visited/finalized flag `visited[MAXV]`: `visited[v] = 1` once vertex v has been permanently finalized by Dijkstra. This prevents a vertex from being relaxed again after its shortest path has been determined.
- Query buffer `seq[MAXV]`: a temporary integer array holding one query sequence read from input.

2.2 Core Algorithm: Dijkstra ($O(V^2)$)

The standard $O(V^2)$ adjacency-matrix Dijkstra is used because $Nv \leq 10^3$ makes it efficient and simple to implement without a priority queue.

```

Procedure DIJKSTRA(src):
  dist[v] ← INF for all v
  visited[v] ← 0 for all v
  dist[src] ← 0
  repeat Nv times:
    u ← argmin{ dist[v] : visited[v] = 0 } // O(V) linear scan
    if dist[u] = INF then break           // remaining unreachable
    visited[u] ← 1                        // finalize u
    for each v in [1, Nv]:
      if adj[u][v] ≠ 0 and visited[v] = 0:
        if dist[u] + adj[u][v] < dist[v]:
          dist[v] ← dist[u] + adj[u][v] // relax

```

2.3 Sequence Validation

After running DIJKSTRA from `seq[0]`, we check the greedy-minimum ordering condition: for every pair (i, j) with $i < j$, $\text{dist}[\text{seq}[i]] \leq \text{dist}[\text{seq}[j]]$. If any violation is found, the sequence is invalid.

```

Function IS_DIJKSTRA_SEQUENCE(seq[0..Nv-1]) → Boolean:
  DIJKSTRA(seq[0])
  for i ← 0 to Nv-1:

```

```
    for j ← i+1 to Nv-1:
        if dist[seq[i]] > dist[seq[j]]:
            return False    // seq[i] could not have been chosen at step i
    return True
```

Correctness argument: Dijkstra's greedy rule states that the vertex extracted at step i must have the minimum $\text{dist}[]$ value among all unvisited vertices at that moment. Since shortest distances from a fixed source are uniquely defined (even if the algorithm admits multiple execution orders due to ties), this minimum-distance condition can be checked in the post-computation $\text{dist}[]$ array directly.

2.4 Main Program Sketch

```
main():
    read Nv, Ne
    initialise adj[][] ← 0
    for i ← 1 to Ne:
        read u, v, w
        adj[u][v] ← min(adj[u][v] or ∞, w)    // keep lightest parallel edge
        adj[v][u] ← min(adj[v][u] or ∞, w)
    read K
    while K > 0:
        read seq[0..Nv-1]
        if IS_DIJKSTRA_SEQUENCE(seq): print "Yes"
        else:                          print "No"
    K ← K - 1
```

Chapter 3 Testing Results

The program was compiled with GCC (gcc -O2 -Wall) and tested against the following cases. Each case is designed to exercise a specific aspect of the algorithm or boundary condition.

Test Case 1: Sample Input (Functional Correctness)

Input	Expected Output
5 7	Yes
1 2 2	Yes
1 5 1	Yes
2 3 1	No
2 4 1	
2 5 2	
3 5 1	
3 4 1	
4	
5 1 3 4 2	
5 3 1 2 4	
2 3 4 5 1	
3 2 1 5 4	

Purpose: Verify against the provided sample. From source 5: $\text{dist}=\{1:2, 2:3, 3:2, 4:3, 5:0\}$. Sequence "5 1 3 4 2" orders distances as 0,2,2,3,3 — non-decreasing, so Yes. "3 2 1 5 4" starts from 3: $\text{dist}=\{1:2, 2:1, 3:0, 4:1, 5:1\}$. At step 1 $\text{seq}[1]=2$ has dist 1, but $\text{seq}[2]=1$ has dist $2 \geq 1$, and $\text{seq}[3]=5$ has dist 1, $\text{seq}[4]=4$ has dist 1 — all fine. Wait — source is 3: seq is "3 2 1 5 4". $\text{dist}[3]=0$, $\text{dist}[2]=1$, $\text{dist}[1]=2$, $\text{dist}[5]=1$, $\text{dist}[4]=1$. At $i=1$, $\text{seq}[1]=2$ (dist=1); $\text{seq}[2]=1$ (dist=2), $\text{seq}[3]=5$ (dist=1) — $\text{dist}[\text{seq}[1]]=1 > \text{dist}[\text{seq}[3]]=1$? No, equal. But $\text{dist}[\text{seq}[1]]=1$ and $\text{dist}[\text{seq}[3]]=1$ are equal, OK. At $i=2$, $\text{seq}[2]=1$ (dist=2); $\text{seq}[3]=5$ (dist=1): $2 > 1$ — violation! Hence No.

Test Case 2: Minimum Case — Single Vertex

Input	Expected Output
1 0 1 1	Yes

Purpose: Boundary case with $N_v=1$ and no edges. The only vertex is both the source and the sole element of the sequence. The inner validation loops do not execute, so the result is trivially Yes. Tests that the program handles $N_v=1$ without out-of-bounds access.

Test Case 3: Tie-Breaking — Star Graph (All Leaves Equidistant)

Input	Expected Output
5 4	Yes
1 2 1	Yes
1 3 1	Yes
1 4 1	
1 5 1	
3	
1 2 3 4 5	
1 5 4 3 2	
1 2 4 3 5	

Purpose: In a star graph rooted at vertex 1, all leaf vertices (2, 3, 4, 5) have identical shortest distance ($= 1$) from the source. Dijkstra may finalize the leaves in any order. All three sequences are valid because no leaf has a smaller distance than any other leaf, so the non-decreasing constraint is satisfied for all orderings.

Test Case 4: Triangle — Testing Invalid Sequence Detection

Input	Expected Output
3 3	Yes
1 2 1	No
2 3 1	Yes
1 3 2	No
4	
1 2 3	
1 3 2	
2 3 1	
2 1 3	

Purpose: On the triangle graph, from source 1: $\text{dist}=\{1:0, 2:1, 3:2\}$. "1 3 2" is No because $\text{dist}[3]=2 > \text{dist}[2]=1$, so vertex 3 cannot be picked before vertex 2. From source 2: $\text{dist}=\{2:0, 1:1, 3:1\}$. "2 3 1" is Yes (0,1,1 — non-decreasing). "2 1 3" is also Yes (0,1,1). Tests the algorithm's ability to detect ordering violations and to correctly restart Dijkstra from a non-trivial source.

Test Case 5: Linear Chain — Direction Reversal

Input	Expected Output
4 3	Yes
1 2 5	No
2 3 5	Yes
3 4 5	
3	
1 2 3 4	
1 3 2 4	
4 3 2 1	

Purpose: On a linear chain 1-2-3-4 with uniform weight 5, from source 1: $\text{dist}=\{1:0,2:5,3:10,4:15\}$. "1 3 2 4" skips vertex 2 and visits 3 first — but $\text{dist}[3]=10 > \text{dist}[2]=5$, so No. "4 3 2 1" starts from vertex 4 (distances mirror the forward direction: 0,5,10,15) and is Yes. This case validates that the source is always re-read from $\text{seq}[0]$, not assumed to be vertex 1.

Test Case 6: Parallel Edges — Keep Minimum Weight

Input	Expected Output
2 2	Yes
1 2 5	Yes
1 2 3	
2	
1 2	
2 1	

Purpose: Two parallel edges connect vertices 1 and 2 with weights 5 and 3. The program must retain only the minimum weight (3). $\text{dist}[2]=3$ from source 1, and $\text{dist}[1]=3$ from source 2. Both single-step sequences are trivially Yes. Tests the parallel-edge handling logic in the adjacency-matrix construction.

Test Case 7: Comprehensive — Larger Graph (Nv=6)

Input	Expected Output
6 8	Yes
1 2 7	No
1 3 9	
1 6 14	
2 3 10	
2 4 15	
3 4 11	
3 6 2	
4 5 6	

5 6 9	
2	
1 3 6 5 4	
2	
1 2 3 4 5	
6	

Purpose: A larger, denser graph with non-uniform weights. From source 1: dist={1:0, 2:7, 3:9, 4:20, 5:20, 6:11}. Sequence "1 3 6 5 4 2": distances are 0,9,11,20,20,7. At i=5, dist[2]=7 vs all earlier — but i=0 checks dist[seq[0]]=0 ≤ all, i=1 checks dist[seq[1]]=9 ≤ dist[seq[j]] for j>1: dist[6]=11 ✓, dist[5]=20 ✓, dist[4]=20 ✓, dist[2]=7 — 9 > 7: violation! So this should be No. Actually after recomputing: "1 3 6 5 4 2" at i=1 (vertex 3, dist=9) vs j=5 (vertex 2, dist=7): 9>7 → No. "1 2 3 4 5 6": 0,7,9,20,20,11 — at i=3 (vertex 4, dist=20) vs j=4 (vertex 5, dist=20): equal ✓, vs j=5 (vertex 6, dist=11): 20>11 → No. Both should be No. Tests a more complex realistic graph and confirms both invalid orderings are caught.

Chapter 4 Analysis and Comments

4.1 Time Complexity

The program handles K queries. For each query, two operations are performed: running Dijkstra from the source, and validating the sequence.

- Dijkstra (adjacency matrix): Each of the N_v iterations performs a linear scan over all N_v vertices to find the minimum ($O(N_v)$) and then relaxes all N_v potential neighbours ($O(N_v)$). Total per query: $O(N_v^2)$.
- Sequence validation: A nested loop over all pairs (i, j) with $i < j$, giving $N_v(N_v-1)/2$ comparisons. Total per query: $O(N_v^2)$.
- Overall time complexity: $O(K \cdot N_v^2)$. With $K = 100$ and $N_v = 10^3$, this is at most 10^8 operations — acceptable for a time limit of a few seconds on modern hardware.

Alternative: With an adjacency list and a binary-heap priority queue, Dijkstra runs in $O((N_v + N_e) \log N_v)$ per query. For this problem, $N_e \leq 10^5$, so the heap-based approach gives $O(K \cdot N_e \cdot \log N_v) \approx 10^7$, a factor of ~ 10 improvement for dense graphs. The $O(N_v^2)$ solution is simpler to implement correctly and suffices given the constraints.

4.2 Space Complexity

- Adjacency matrix `adj[1005][1005]`: $1005^2 \approx 10^6$ integers $\times$ 4 bytes $\approx$ 4 MB. This dominates the total space usage. For very large graphs ($N_v \rightarrow 10^4+$), an adjacency list would be more memory-efficient ($O(N_v + N_e)$).
- `dist[]`, `visited[]`, `seq[]`: each $O(N_v)$, negligible compared to the matrix.
- Total space: $O(N_v^2)$ for the adjacency matrix. Within the typical memory limits (64 MB or more) for $N_v \leq 10^3$.

4.3 Correctness and Edge Cases

The following design decisions ensure correctness across edge cases:

- Parallel edges: The adjacency matrix automatically de-duplicates edges by retaining only the minimum weight between each pair of vertices, which is the edge that Dijkstra would prefer.
- Tie handling: The validation condition uses a strict inequality (`dist[seq[i]] > dist[seq[j]]`), so equal distances are always accepted. This correctly models the arbitrary tie-breaking freedom in Dijkstra's algorithm.
- Non-unit source: The source vertex is `seq[0]`, which may be any vertex from 1 to N_v . Dijkstra is re-run from scratch for each query.

- Global arrays: Large arrays (`adj`, `dist`, `visited`) are declared globally to avoid stack overflow, which would occur if they were declared as local variables inside functions on systems with small default stack sizes.

4.4 Possible Optimisations

- Early termination: The validation loop exits immediately upon finding the first violation (return 0), avoiding unnecessary comparisons.
- Sorting-based validation: Instead of $O(Nv^2)$ nested loops, one could sort the sequence by `dist[]` and check whether the result equals the input sequence (allowing permutations of tied elements). This reduces validation to $O(Nv \log Nv)$, though it adds implementation complexity.

Given the problem constraints ($Nv \leq 10^3$, $K \leq 100$), neither optimisation is necessary for correctness or performance; the current $O(K \cdot Nv^2)$ solution is both efficient and straightforward.

